

One server, many sites

Nginx as it ships, then as the exercises leave it. Every block below came out of a running lab: the process tree, the sockets it holds, the config it will accept, and the requests it answered.

Command `qlab run nginx-lab`

VM `nginx-lab`, port 80 forwarded to a dynamic host port

Credentials `labuser` / `labpass`

Outcome `qlab test nginx-lab` → 6 exercises, 36 checks, all passed

1. What is running

```
nginx version: nginx/1.18.0 (Ubuntu)
active
enabled
```

```
root      1605      1 nginx: master process /usr/sbin/nginx -g daemon on; master_process on;
www-data  2230     1605 nginx: worker process
```

Two processes, and the split between them is the whole architecture. The **master** runs as `root` — it has to, because binding port 80 requires privilege and because it is the one that re-reads the configuration. The **worker** runs as `www-data` and is the process that actually talks to the network. A flaw in request handling therefore lands in an unprivileged process, which is the entire point of the arrangement.

That is also why `systemctl reload nginx` is not the same as `restart`: the master keeps the listening sockets and hands the workers a new configuration, so no connection is dropped.

```
LISTEN 0      4096      127.0.0.53%lo:53      0.0.0.0:*      users:((("systemd-resolve",pid=210,fd=14))
LISTEN 0      511        0.0.0.0:80          0.0.0.0:*      users:((("nginx",pid=2230,fd=6),
("nginx",pid=1605,fd=6))
LISTEN 0      128        0.0.0.0:22          0.0.0.0:*      users:((("sshd",pid=1664,fd=3))
LISTEN 0      511        [::]:80            [::]:*         users:((("nginx",pid=2230,fd=7),
("nginx",pid=1605,fd=7))
LISTEN 0      128        [::]:22            [::]:*         users:((("sshd",pid=1664,fd=4))
```

2. The site it boots with

```
total 0
lrwxrwxrwx 1 root root 34 Sep 14 09:27 default -> /etc/nginx/sites-available/default

default
```

`sites-available` holds what exists; `sites-enabled` holds what is switched on, as symlinks. Disabling a site is removing a link, not deleting a file — which is why the two directories exist at all.

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    root /var/www/html;
    index index.html index.htm index.nginx-debian.html;
    server_name _;
    location / {
        try_files $uri $uri/ =404;
    }
}
```

Three things are worth naming here. `default_server` makes this the site that answers when nothing else matches. `server_name _` is not a wildcard with meaning — it is deliberately an invalid hostname, so this block only ever gets requests by being the default. And `try_files $uri $uri/ =404` says: try the file, then the directory, then give up honestly, rather than falling through to something unintended.

3. Nginx will not load a configuration it dislikes

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

`nginx -t` parses the whole tree and answers before anything is live. It is the command to run before every reload; the failure mode it prevents is a typo that takes the server down at the moment you reload it.

4. Requests, and where an unknown name lands

```
$ curl -I http://localhost/
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Mon, 14 Sep 2026 09:32:12 GMT
Content-Type: text/html
Content-Length: 19
Last-Modified: Mon, 14 Sep 2026 09:31:55 GMT

$ curl -H "Host: mysite.local" http://localhost/
<h1>Test Page</h1>
```

The second request is the interesting one. It asks for `mysite.local`, a name this server has no site for — the exercises create that virtual host and then remove it again. With no match, the request is answered by the block marked `default_server`, which is why it returns the default page instead of an error.

On a real server that is a trap worth knowing: a hostname pointed at your address that you never configured does not get refused, it gets whatever your default site serves.

```
127.0.0.1 - - [14/Sep/2026:09:32:10 +0000] "GET /secret/ HTTP/1.1" 401 188 "-" "curl/7.81.0"
127.0.0.1 - admin [14/Sep/2026:09:32:10 +0000] "GET /secret/ HTTP/1.1" 200 16 "-" "curl/7.81.0"
127.0.0.1 - admin [14/Sep/2026:09:32:10 +0000] "GET /secret/ HTTP/1.1" 200 16 "-" "curl/7.81.0"
127.0.0.1 - admin [14/Sep/2026:09:32:10 +0000] "GET /secret/ HTTP/1.1" 401 188 "-" "curl/7.81.0"
127.0.0.1 - - [14/Sep/2026:09:32:12 +0000] "HEAD / HTTP/1.1" 200 0 "-" "curl/7.81.0"
127.0.0.1 - - [14/Sep/2026:09:32:12 +0000] "GET / HTTP/1.1" 200 19 "-" "curl/7.81.0"
```

Every line in `access.log` is one request: address, time, request line, status, bytes, referrer and user agent. The format is set by `log_format` and the default is called `combined`.

5. Verification

```
Exercise 1: All 9 checks passed
Exercise 2: All 6 checks passed
Exercise 3: All 4 checks passed
Exercise 4: All 7 checks passed
Exercise 5: All 4 checks passed
Exercise 6: All 6 checks passed
Exercises run:      6
Exercises passed:   6
Exercises failed:   0
Exercises skipped:  0
All exercises passed!
```

The six exercises build and tear down what this document only shows at rest: a second virtual host, a reverse proxy in front of a local application, log analysis, and the security headers.

6. What to take away

- Master as root, workers as `www-data` : privilege only where it is needed.
- `reload` re-reads the configuration without dropping connections; `restart` drops them.
- `sites-available` and `sites-enabled` separate *existing* from *enabled*, with a symlink as the switch.
- `default_server` catches every name you did not configure. It is not an error path, it is a fallback — check what it serves.
- Run `nginx -t` before every reload.

`guide.md` in the plugin carries the exercises: virtual hosts, reverse proxying, reading the logs, and the first security headers.